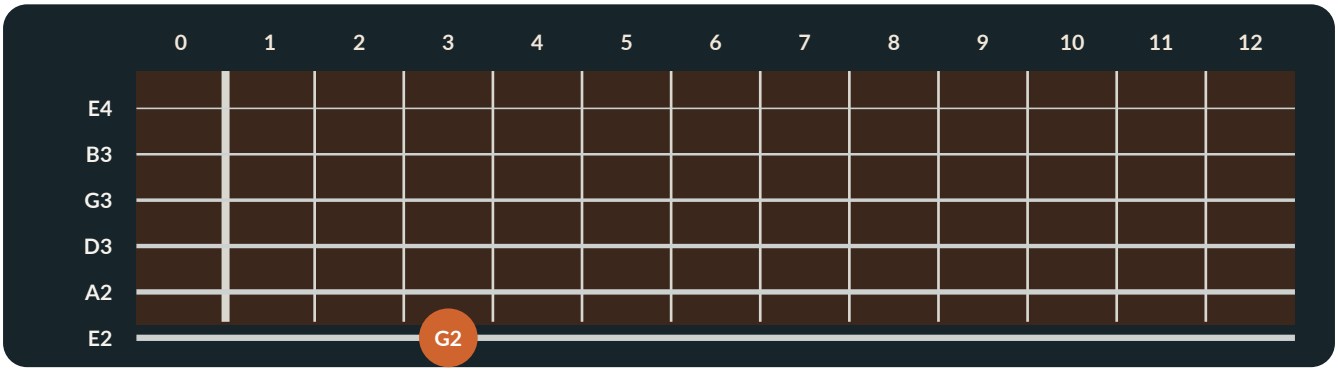
CHAPITRE 01 / OCTOBRE 2026

MODAL GUITAR LAB

Du projet Vite au premier manche interactif

Carnet de développement & guide de reconstruction pas à pas



Objectif de cette séance. Afficher les six cordes d'une guitare standard, représenter les positions 0 à 12, sélectionner une case au clic et afficher sa note complète et sa hauteur MIDI.

Statut : incrément terminé et validé manuellement. **Périmètre exclu :** audio, tablature, composition, backend, authentification.

Nature du document. Cette documentation retrace les décisions et exercices de la séance, puis fournit une **version consolidée de reconstruction** des fichiers. Il ne s'agit pas d'une capture automatique du dépôt local : quelques extraits ont été harmonisés pour former un ensemble cohérent et reproductible.

01. L'application, son intention et ses limites

Modal Guitar Lab est un petit laboratoire musical local dans le navigateur, inspiré de Guitar Pro mais centré sur **l'exploration des gammes, des modes et la composition de solos de guitare**. L'idée est de pouvoir, à terme, choisir une progression harmonique, entendre un accompagnement, visualiser les notes sur un manche, éditer un solo et le rejouer.

Une progression comme **Em | C | Bm | F** est un exemple musical, pas une contrainte : l'application doit encourager la recherche des couleurs harmoniques, des tensions et des résolutions, sans forcer à rester dans une seule gamme.

L'objectif n'est pas de construire un produit commercial. Ce projet est un **loisir technique et musical** qui permet de réactualiser React et TypeScript en codant soi-même. Il reste facultatif et indépendant de CarbonTrace et du parcours professionnel.

Du projet rêvé à un premier incrément vérifiable

À terme — pistes possibles	Séance 1 — réalisé
Progressions, gammes, modes et accompagnement	Manche à six cordes en accordage standard
Tablature synchronisée et édition de solos	Grille de 13 positions par corde, 0 à 12
Lecture, bouclage et export MIDI éventuel	Clic sur une position et surlignage visuel
Accordages alternatifs et couleurs harmoniques	Nom, octave et numéro MIDI de la note sélectionnée

Critères de réussite réellement validés

- Une position correspond à une paire **(numéro de corde, numéro de case)**.
- Les cordes à vide sont correctement traitées : **case 0** est une position valide.
- Une seule position est surlignée ; la sortie textuelle suit la sélection.
- Le build Vite/TypeScript passe avec les contrôles de typage activés.

Le document de démarrage du projet reste la source de référence de la vision générale. Ce chapitre détaille uniquement ce premier incrément.

02. Choix techniques et raisons

Choix	Pourquoi ce choix ?
React + TypeScript	Réviser composants, JSX, événements, hooks, unions de types et vérifications statiques.
Vite + pnpm	Initialisation rapide et environnement de développement simple.
Variant TypeScript classique	Ne pas ajouter React Compiler tant que l'optimisation n'apporte rien au projet.
strict + noUncheckedIndexedAccess	Détecter davantage d'erreurs de types, notamment les valeurs éventuellement undefined.
CSS natif + Grid	Conserver la maîtrise du rendu des cordes et cases sans bibliothèque UI.
Hauteur MIDI comme base	Une case ajoute un demi-ton à la corde à vide ; nom et octave deviennent calculables.
useState pour la seule sélection	Éviter de dupliquer une note, une octave et un numéro MIDI comme états indépendants.
music/notes.ts	Isoler les calculs réutilisables et indépendants de React.

Schéma du flux de données

Flux de sélection et de calcul

```
Clic sur une case (corde 6, case 3)
          |
          v
setSelectedPosition({ stringPos: 6, fret: 3 })
          |
          v
React effectue un nouveau rendu
          |
          +--> .find() => openMidi = 40
          |                |
          |                v
          |        getMidiNote(40, 3) = 43
          |                |
          |                v
          |        getFullNoteName(43) = "G2"
          |
          +--> cercle selectionne + "G2" + "MIDI 43"
```

Le **seul état utilisateur nécessaire** à cette étape est la paire corde/case. Le reste est déduit au rendu. Cela évite qu'une sélection indique une case et qu'un autre état affiche une note obsolète.

03. Déroulé de la séance — pas à pas

Étape A — Partir d'un nouveau projet Vite

Après avoir envisagé la réutilisation d'une ancienne base React, nous avons choisi un démarrage propre. Dans l'assistant Vite, le variant retenu est **React + TypeScript**, sans React Compiler. Le gestionnaire de paquets est **pnpm**.

Commandes de démarrage

```
pnpm create vite modal-guitar-lab --template react-ts
cd modal-guitar-lab
pnpm install
pnpm dev
```

Le serveur local lance la page de démonstration. La modification d'un composant est reflétée grâce au **file watching** de Vite, à la mise à jour des modules (**HMR**) et à **React Fast Refresh**, qui conserve souvent l'état pendant les modifications.

Nous avons distingué deux notions : **TypeScript strict** renforce les vérifications à la compilation ; **React StrictMode** est un outil de diagnostic React en développement. Ils sont indépendants.

Étape B — Épurer le point d'entrée

La démonstration Vite a été retirée. Le composant **App** ne conserve qu'un contenu principal sémantique, un titre puis l'appel à **GuitarNeck**. Les styles globaux sont réduits à **box-sizing: border-box**, une marge nulle sur le body et une police système.

Cette étape a servi à revoir les composants fonctionnels, le JSX, les imports et la différence entre **<main>** (contenu principal) et **<section>** (section thématique). Un fragment React n'est pas nécessaire avec un unique élément racine.

Étape C — Modéliser les six cordes

La première version représentait une corde avec **pos**, **name** et **octave**. Le rendu utilisait **.map()** et une propriété **key** stable. Une erreur initiale sur la corde de La a été corrigée : la cinquième corde à vide est **A2**, et non A3.

Corde	Note à vide	MIDI
1 — aiguë	E4	64
2	B3	59
3	G3	55
4	D3	50
5	A2	45
6 — grave	E2	40

Les cordes sont affichées **de l'aiguë vers la grave**, comme sur une tablature, même si l'on énonce souvent l'accordage standard de la grave vers l'aiguë (E–A–D–G–B–E).

Étape D — Dessiner les cordes en CSS

Chaque ligne de corde est d'abord une rangée Flexbox. La corde occupe l'espace restant grâce à **flex: 1**, plutôt qu'à **width: 100%**, qui ne tient pas directement compte de la largeur du nom de note. Une variable CSS **--i** transporte le numéro de corde afin de moduler son épaisseur.

Propriété CSS personnalisée

```
style={{ '--i': string.pos } as React.CSSProperties}

/* Plus la corde est grave, plus elle est epaisse. */
.string-line {
  height: calc(1px + var(--i) * 0.6px);
}
```

Étape E — Créer 13 positions par corde

Nous avons retenu les cases **1 à 12** et ajouté **0** pour la corde à vide : cela fait **13 positions × 6 cordes = 78 cellules**. Le tableau des positions est construit dynamiquement :

Génération des numéros de cases

```
const fretNumbers = Array.from({ length: 13 }, (_, i) => i);
```

Une boucle **.map()** sur les cordes contient une seconde boucle **.map()** sur les frettes. Chaque ligne reçoit **key={string.pos}** et chaque cellule **key={fret}**. Les clés sont uniques parmi leurs éléments frères ; elles ne doivent pas l'être dans toute l'application.

Étape F — Aligner grille et en-tête

CSS Grid distribue chaque ligne en **une colonne fixe de 50 px** (nom de corde) et **13 colonnes de largeur égale**. L'en-tête doit contenir une cellule vide avant les numéros : sans elle, la valeur 0 se place dans la colonne du nom de corde et toutes les étiquettes se décalent.

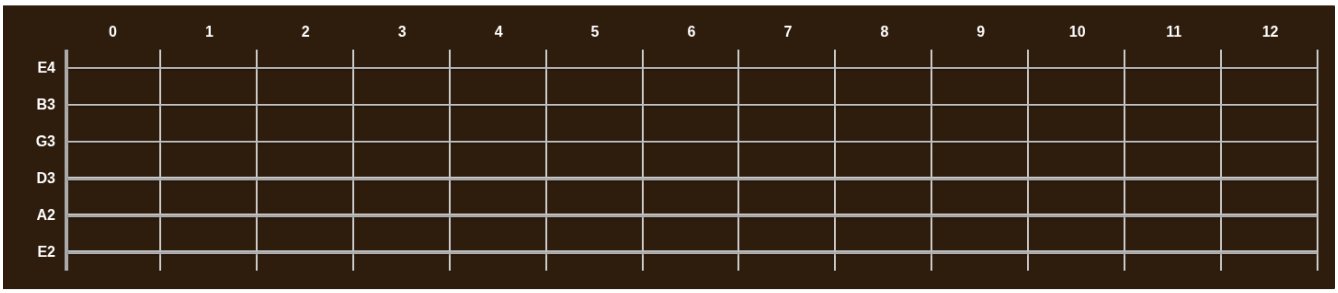
Structure de colonnes identique

```
.string-row,
.fretboard-headers {
  display: grid;
  grid-template-columns: 50px repeat(13, 1fr);
}
```

Le sillet se situe **entre la position à vide 0 et la case 1**. Le sélecteur **.string-name + .fret-cell** cible la première cellule immédiatement après le nom de corde et lui ajoute une bordure droite épaisse. **:first-of-type** était trompeur car l'élément précédent, lui aussi un div, est du même type HTML.

Capture intermédiaire de la séance

Modal-guitar-lab



Capture fournie pendant la séance : cordes, numéros, espacement et frettes alignés, avant l'ajout de l'étiquette de note dans le cercle de sélection.

Étape G — Associer un clic à des coordonnées

Chaque cellule reçoit un **onClick** qui appelle une fonction avec le numéro de corde et le numéro de case. L'écriture fléchée reporte l'appel au moment du clic au lieu d'exécuter immédiatement la fonction pendant le rendu.

Gestion de l'événement

```
onClick={() => handleFretClick(string.pos, fret)}
```

Le premier contrôle se faisait dans la console. Des essais sur **(1, 0)**, **(6, 3)** et **(2, 12)** ont confirmé l'association entre chaque zone et ses coordonnées.

Étape H — Mémoriser la sélection avec useState

Une position sélectionnée contient deux propriétés qui forment un tout. L'état initial est **null** : aucune note n'est sélectionnée au démarrage.

Un objet d'état, pas deux états séparés

```
type SelectedPosition = {
  stringPos: GuitarString['pos'];
  fret: number;
};

const [selectedPosition, setSelectedPosition] =
  useState<SelectedPosition | null>(null);
```

Le type **GuitarString['pos']** est un **Indexed Access Type** : on réutilise le type de la propriété **pos**. Le rendu conditionnel vérifie l'existence de l'objet sélectionné, jamais la véracité de **fret** : une case **0** est valide mais **falsy** en JavaScript.

Étape I — Surligner une seule cellule

Une fonction **cellIsSelected()** compare à la fois la corde et la case. Une classe **is-selected** est ajoutée conditionnellement. Un pseudo-élément CSS **::after** trace un disque coloré au milieu de la cellule. Cette représentation est **dérivée** de selectedPosition : aucun état additionnel n'est créé.

Étape J — Construire la logique musicale hors de React

La seconde moitié de la séance a porté sur un module **src/music/notes.ts**. L'idée directrice est simple : la hauteur MIDI de la note = **hauteur MIDI de la corde à vide + numéro de case**. La note chromatique se calcule avec le modulo 12 et l'octave avec une division entière.

Les trois formules musicales

```
midi = openStringMidi + fret
indexChromatique = midi % 12
octave = Math.floor(midi / 12) - 1
```

La convention retenue est **MIDI 60 = C4** ; MIDI 0 correspond donc à C-1. Les noms de notes utilisent provisoirement les dièses (C#, D#, F#, G#, A#). Les bémols enharmoniques ne sont pas encore traités.

Étape K — Vérifier les indices et les entrées

Nous avons activé **noUncheckedIndexedAccess**. Après un accès à un tableau par un index calculé, TypeScript considère que le résultat peut être **undefined**. La fonction vérifie explicitement le résultat et lance une erreur en cas d'anomalie, sans assertion forcée.

Nous avons aussi factorisé la validation MIDI dans **checkMidiRange()** : entier compris entre **0 et 127**. Le nom de note et l'octave utilisent cette validation. Le code de getMidiNote lui-même reste la simple addition de la séance ; son domaine d'entrée est pour le moment contrôlé par les données du manche.

Étape L — Relier les calculs à l'interface

À la sélection, **.find()** retrouve la corde correspondant à selectedPosition.stringPos. Cette opération peut retourner undefined : le rendu vérifie donc que la sélection **et** la corde existent avant de calculer l'affichage.

Le nom complet (ex. **G2**) et le numéro MIDI sont montrés sous le manche. Enfin, **data-note** fournit le nom de la note au pseudo-élément **::after** pour l'afficher dans le disque. Le calcul n'est effectué pour cette étiquette que sur la cellule sélectionnée.

Étape M — Simplifier le modèle final

Au départ, chaque corde stockait son nom et son octave. Une fois getFullNoteName disponible, ces valeurs devenaient dérivables d'**openMidi**. Le type a été simplifié :

Modèle retenu en fin de séance

```
type GuitarString = {
  pos: 1 | 2 | 3 | 4 | 5 | 6;
  openMidi: number;
};
```

Cela rend plus simple un changement ultérieur d'accordage, sans l'implémenter encore. La séance a également clarifié que **useMemo** évite éventuellement des recalculs lorsque ses dépendances n'ont pas changé ; il n'apporte rien ici pour de simples additions et indexations.

04. Reconstruction complète — fichiers de référence

Les extraits suivants constituent une **version consolidée et cohérente** du premier incrément. Ils reprennent la structure et les comportements validés dans l'échange, avec un nettoyage mineur (noms, mise en forme, calcul uniquement sur la cellule sélectionnée). Ils sont destinés à une reproduction pédagogique et ne prétendent pas reproduire octet pour octet les fichiers locaux.

Arborescence minimale

Organisation des fichiers

```
modal-guitar-lab/
├── src/
│   ├── music/
│   │   └── notes.ts
│   ├── GuitarNeck.tsx
│   ├── App.tsx
│   ├── App.css
│   ├── index.css
│   └── main.tsx       # fichier de demarrage Vite conserve
├── tsconfig.app.json
├── package.json
└── ...
```

4.1 — Configuration TypeScript

Dans **tsconfig.app.json**, conserver les autres paramètres générés par Vite et vérifier la présence de ces deux options dans **compilerOptions** :

Extrait des options à conserver ou ajouter

```
"compilerOptions": {
  "strict": true,
  "noUncheckedIndexedAccess": true
}
```

Ne pas remplacer tout le contenu généré du fichier par cet extrait : il ne montre que les options discutées. Selon la version du template, l'option strict peut déjà être activée.

4.2 — src/music/notes.ts

Fichier complet : notes.ts

```
export function getMidiNote(
  openStringMidi: number,
  fretNumber: number
): number {
  return openStringMidi + fretNumber;
}

export function checkMidiRange(midi: number): void {
  if (!Number.isInteger(midi) || midi < 0 || midi > 127) {
    throw new RangeError(
      'MIDI note must be an integer between 0 and 127'
    );
  }
}

export function getNoteName(midi: number): string {
  checkMidiRange(midi);
  const NOTES = [
    'C', 'C#', 'D', 'D#', 'E', 'F',
    'F#', 'G', 'G#', 'A', 'A#', 'B'
  ];

  const note = NOTES[midi % NOTES.length];
  if (note === undefined) {
    throw new TypeError('Invalid note index');
  }
  return note;
}

export function getOctave(midi: number): number {
  checkMidiRange(midi);
  return Math.floor(midi / 12) - 1;
}

export function getFullNoteName(midi: number): string {
  return `${getNoteName(midi)}${getOctave(midi)}`;
}
```

Pourquoi ce découpage ?

- **getMidiNote** : traduit une coordonnée physique de guitare en hauteur numérique.
- **getNoteName** : transforme la hauteur en classe de note chromatique, sans octave.
- **getOctave** : détermine le registre du son selon la convention C4 = 60.
- **getFullNoteName** : compose la présentation musicale, par exemple C#4.
- **checkMidiRange** : centralise les contraintes d'entrée MIDI utilisées par ces fonctions.

4.3 — src/GuitarNeck.tsx : types et logique

Début du fichier : types et fonctions

```
import { useState, type CSSProperties } from 'react';
import { getFullNoteName, getMidiNote } from './music/notes';

type GuitarString = {
  pos: 1 | 2 | 3 | 4 | 5 | 6;
  openMidi: number;
};

type SelectedPosition = {
  stringPos: GuitarString['pos'];
  fret: number;
};

function cellIsSelected(
  pos: GuitarString['pos'],
  fret: number,
  selected: SelectedPosition | null
): boolean {
  return selected !== null &&
    selected.stringPos === pos &&
    selected.fret === fret;
}

function buildOutput(position: SelectedPosition, string: GuitarString) {
  const midi = getMidiNote(string.openMidi, position.fret);
  const note = getFullNoteName(midi);
  return <span>{`NOTE: ${note} MIDI: ${midi}`}</span>;
}
```

4.4 — src/GuitarNeck.tsx : composant

Suite du même fichier : composant complet

```
function GuitarNeck() {
  const [selectedPosition, setSelectedPosition] =
    useState<SelectedPosition | null>(null);

  const guitarStrings: GuitarString[] = [
    { pos: 1, openMidi: 64 },
    { pos: 2, openMidi: 59 },
    { pos: 3, openMidi: 55 },
    { pos: 4, openMidi: 50 },
    { pos: 5, openMidi: 45 },
    { pos: 6, openMidi: 40 }
  ];

  const fretNumbers = Array.from({ length: 13 }, (_, i) => i);

  const handleFretClick = (
    stringPos: GuitarString['pos'], fret: number
  ) => {
    setSelectedPosition({ stringPos, fret });
  };

  const selected = guitarStrings.find(
    string => string.pos === selectedPosition?.stringPos
  );

  return (
    <>
      <div className="g-neck-container">
        <div className="fretboard-headers">
          <div className="header-cell empty" />
          {fretNumbers.map(fret => (
            <div key={fret} className="header-cell">{fret}</div>
          ))}
        </div>

        <div className="fretboard-grid">
          {guitarStrings.map(string => (
            <div key={string.pos} className="string-row">
              <div className="string-name">
                {getFullNoteName(string.openMidi)}
              </div>

              {fretNumbers.map(fret => {
                const isSelected = cellIsSelected(
                  string.pos, fret, selectedPosition
                );
                const note = isSelected
                  ? getFullNoteName(getMidiNote(string.openMidi, fret))
                  : '';

                return (
                  <div
                    key={fret}
                    className={`fret-cell ${isSelected ? 'is-selected' : ''}`}
                    onClick={() => handleFretClick(string.pos, fret)}
                    data-note={note}
                  >
                    <div
                      className="string-line"
                      style={{ '--i': string.pos } as CSSProperties}
                    />
```

suite du même fichier

```
                  </div>
                );
              })}
            </div>
          ))}
        </div>
      </div>

      {selectedPosition && selected
        ? buildOutput(selectedPosition, selected)
        : <span>Aucune note selectionnee</span>}
    </>
  );
}

export default GuitarNeck;
```

Ce JSX conserve les cellules **div** cliquables utilisées pendant la séance. Ce choix a permis de concentrer l'apprentissage sur React et CSS Grid ; l'accessibilité au clavier reste à traiter ultérieurement. Pour obtenir un comportement accessible, on envisagera des boutons avec leurs libellés et styles appropriés.

4.5 — src/App.tsx et src/index.css

App.tsx

```
import './App.css';
import GuitarNeck from './GuitarNeck';

function App() {
  return (
    <main>
      <h1>Modal Guitar Lab</h1>
      <GuitarNeck />
    </main>
  );
}

export default App;
```

index.css

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
}
```

4.6 — src/App.css : grille et apparence

Première partie : bois, grille et sillet

```
.g-neck-container {
  background: #2e1c0c;
  padding: 20px;
  color: white;
  border-radius: 4px;
  overflow-x: auto;
}

.fretboard-headers {
  display: grid;
  grid-template-columns: 50px repeat(13, minmax(0, 1fr));
  text-align: center;
  margin-bottom: 10px;
  font-weight: bold;
}

.fretboard-grid {
  display: flex;
  flex-direction: column;
}

.string-row {
  display: grid;
  grid-template-columns: 50px repeat(13, minmax(0, 1fr));
  height: 40px;
  align-items: center;
}

.string-name {
  font-weight: bold;
  padding-right: 10px;
  text-align: right;
}

.fret-cell {
  position: relative;
  height: 100%;
  border-right: 2px solid #ccc;
  cursor: pointer;
}

/* Sillet entre la position 0 et la case 1. */
.string-name + .fret-cell {
  border-right: 4px solid #ddd;
}
```

Seconde partie : cordes et cercle de sélection

```
.string-line {
  position: absolute;
  top: 50%;
  left: 0;
  right: 0;
  transform: translateY(-50%);
  height: calc(1px + var(--i) * 0.6px);
  background: linear-gradient(to bottom, #eee, #999, #bbb);
  box-shadow: 0 1px 2px rgba(0, 0, 0, 0.4);
  pointer-events: none;
}
```

suite du même fichier

```
.is-selected::after {
  content: attr(data-note);
  position: absolute;
  left: 50%;
  top: 50%;
  transform: translate(-50%, -50%);
  z-index: 1;
  width: 29px;
  height: 29px;
  background-color: #ff5722;
  color: white;
  border-radius: 50%;
  font-size: 10px;
  font-weight: bold;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.4);
  pointer-events: none;
}
```

Les valeurs visuelles ci-dessus sont des réglages représentatifs de la séance. L'effet métallique, la couleur bois et les bordures sont ajustables ; l'essentiel pour le fonctionnement est l'identité des colonnes Grid, la relation corde/case et la classe de sélection.

05. Jeu de tests et vérification

Les tests fonctionnels de la séance ont été réalisés manuellement dans le navigateur ; un **pnpm build** réussi a confirmé la compilation. Aucun framework de test automatisé n'a été ajouté.

Action	Résultat attendu
Chargement initial	« Aucune note sélectionnée » ; aucun cercle actif
Corde 6, case 3	G2, MIDI 43 ; cercle sur (6, 3)
Corde 2, case 1	C4, MIDI 60 ; cercle sur (2, 1)
Corde 1, case 0	E4, MIDI 64 ; la corde à vide est acceptée
Corde 1, case 12	E5, MIDI 76
Corde 5, case 12	A3, MIDI 57
Clic sur une autre case	Le cercle précédent disparaît ; un seul reste actif
Dans le terminal	pnpm build se termine sans erreur

Tests de raisonnement de la logique musicale

Appel	Résultat
getMidiNote(40, 3)	43
getNoteName(61)	C#
getOctave(59)	3
getOctave(60)	4
getFullNoteName(76)	E5
getNoteName(-1)	RangeError
getNoteName(128)	RangeError
getNoteName(40.5)	RangeError

Ces derniers cas constituent une **table de référence** pour de futurs tests automatisés ; ils ne signifient pas qu'une suite de tests a été mise en place pendant cette séance.

Erreurs typiques à retrouver si le rendu diverge

- **Les numéros sont décalés d'une colonne :** vérifier la cellule d'en-tête vide.
- **Le sillet est avant la corde à vide :** déplacer la bordure épaisse à droite de la cellule 0.
- **Le clic sur la case 0 n'affiche rien :** tester selectedPosition, pas selectedPosition.fret.
- **TypeScript signale string | undefined :** traiter explicitement le résultat de NOTES[index].
- **La 5e corde donne A3 :** la valeur correcte à vide est A2, MIDI 45.
- **Le cercle affiche toujours « La » :** alimenter data-note avec le résultat du calcul musical.

06. Les acquis techniques — ce qu'il faut savoir refaire

React

- **Composant :** une fonction retourne du JSX ; l'interface est produite depuis les données.
- **.map() et key :** une structure de données devient une série d'éléments React identifiables.
- **Gestionnaire d'événement :** une fonction fléchée permet de transmettre corde et case au clic.
- **useState :** la sélection persiste entre les rendus ; setter et rerender actualisent l'interface.
- **Rendu conditionnel :** l'absence de sélection est représentée par null.
- **Données dérivées :** la note, l'octave, la hauteur MIDI et la classe CSS découlent de l'état.
- **useMemo :** optimiser un calcul coûteux et stable selon ses dépendances, pas mémoriser chaque addition.

TypeScript et JavaScript

- **Unions littérales :** pos est restreint à 1 | 2 | 3 | 4 | 5 | 6.
- **Indexed Access Types :** SelectedPosition reprend le type GuitarString['pos'].

- **Narrowing :** après un contrôle null/undefined, TypeScript affine les types.
- **strict / noUncheckedIndexedAccess :** typage renforcé, sans prétendre garantir les valeurs à l'exécution.
- **Fonctions pures :** les calculs musicaux n'ont pas d'effet de bord.
- **Modulo :** retrouver l'une des 12 classes chromatiques ; **Math.floor :** identifier l'octave.
- **Exceptions :** rejeter les valeurs MIDI non entières ou hors de 0–127.

CSS et présentation

- **Flexbox :** première représentation des cordes et distribution de l'espace libre.
- **CSS Grid :** 50 px + 13 colonnes égales, en-tête et lignes synchronisés.
- **Custom properties :** une variable CSS encode l'épaisseur de chaque corde.
- **Pseudo-élément ::after :** un attribut data-note se transforme en étiquette visuelle.

07. Décisions ouvertes et suites possibles

À la fin de ce chapitre, le manche sélectionne et identifie correctement une note. Les points ci-dessous ne sont **pas** des défauts bloquants ; ce sont des pistes éventuelles, à reprendre selon l'envie et les besoins réels.

Sujet	Situation à la fin de la séance
Accessibilité	Cellules div cliquables ; interaction clavier et libellés accessibles à améliorer.
Tests automatisés	Absents ; les cas manuels et une base de fonctions pures facilitent l'ajout ultérieur.
Affichage mobile	À éprouver ; 13 colonnes peuvent devenir étroites sur un écran réduit.
Altérations	Orthographe des notes en dièses uniquement ; pas de bémols contextuels.
Accordages alternatifs	Non développés, mais représentation openMidi adaptée à cette extension.
Validation de getMidiNote	Addition simple ; le contrôle de plages peut être ajouté si les entrées deviennent externes.
Audio / tablature / édition	Explicitement hors périmètre de cette séance.

Le point de reprise

Prochain chat : *MGL — 02 — Visualisation des gammes sur le manche*. Première idée d'incrément : représenter la gamme de Mi mineur naturel, déterminer si une note en fait partie, puis la visualiser sur le manche. Aucune obligation de faire tout cela d'un seul coup.

Prompt de reprise proposé : « Nous reprenons Modal Guitar Lab. La séance 1 est validée : React + TypeScript strict + Vite + pnpm ; GuitarNeck avec 6 cordes, positions 0–12, sélection avec useState ; notes.ts expose getMidiNote, checkMidiRange, getNoteName, getOctave et getFullNoteName. Les tests manuels et pnpm build passent. Guide-moi pas à pas pour commencer la visualisation de Mi mineur naturel, en me laissant écrire le code. »